

Blockchain +2

How to Create a Blockchain

Blockchain is a distributed ledger that processes transactions across a network of computers. Here's more on how blockchain works, the benefits of using it and a step-by-step guide on how to create a blockchain.



Written by [Sinai Nday](#)

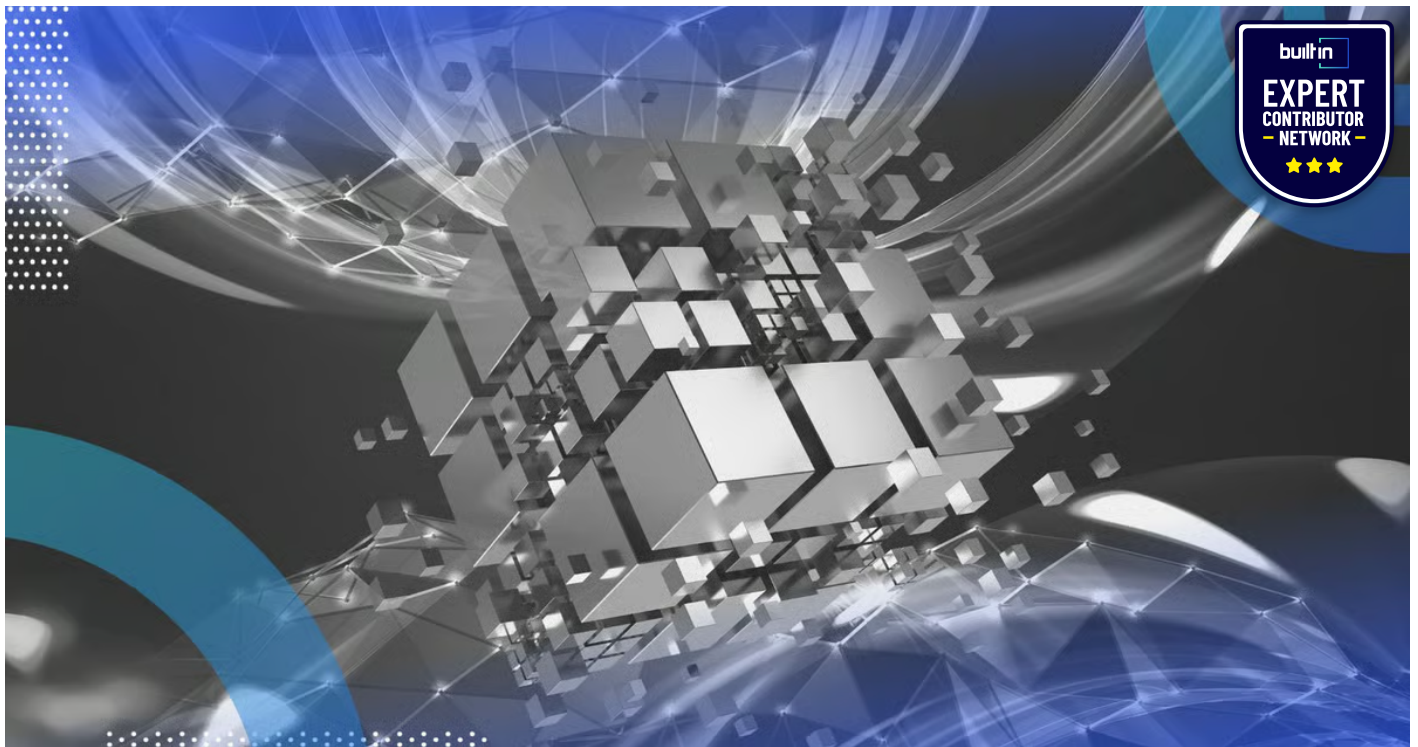


Image: Shutterstock / Built In

UPDATED BY

[Matthew Urwin](#) | Jul 09, 2025

Summary: Creating a blockchain involves linking cryptographically hashed blocks in a decentralized network. Built with languages like Go, Rust or C++, blockchains offer transparency, security and resilience, making them useful in finance, healthcare, real estate and beyond.

[Jobs](#)[Companies](#)[Articles](#)
[Tracker](#)[More](#)

What is [blockchain](#)? Is it hard to build one? Where do you start? Which programming language should I use?

I get these questions quite often when meeting people who are interested in blockchain technology. While there are plenty of blockchain resources online, they can be overwhelming and frustrating to understand. However, this article is a little different than those other resources.

4 Steps to Creating a Blockchain

1. Create a block.
2. Add the data (header and body) to the block.
3. Hash the block.
4. Chain the blocks together.

I'll guide you through the basic concepts of blockchain and how to program one using [Golang \(Go\)](#).

For speed, endurance and security, most blockchain core engines are built in [C/C++](#) (Bitcoin, EOS), Go (Hyperledger Fabric, Ethereum), [Java](#) (Ethereum), [Rust](#), Haskell (Cardano) and [Ruby](#) (Ethereum), then provide bindings to other easy-to-use [programming languages](#).

Also, some blockchain engines combine many [programming languages](#) for robustness and ease of use for developers. Ethereum is the best example of this.

Building a Blockchain in Golang - Part 1

Tensor Programming



A tutorial on how to build a blockchain in Golang. | Video: Tensor Programming

What Prerequisites Do I Need to Create a Blockchain?

- A network.
- [Cryptography](#).
- A data structure and [algorithms](#).
- Decentralized systems.
- Javascript, Go or Python knowledge.

You only need to understand the basic concepts to program your first blockchain prototype, so let's begin with some theories.

MORE ON BLOCKCHAIN

[SushiSwap vs. Uniswap: What Are the Differences?](#)



What Is a Blockchain?

Blockchain is not Bitcoin. Blockchain is not a digital currency. Blockchain is a set of

Jobs

Companies

Articles

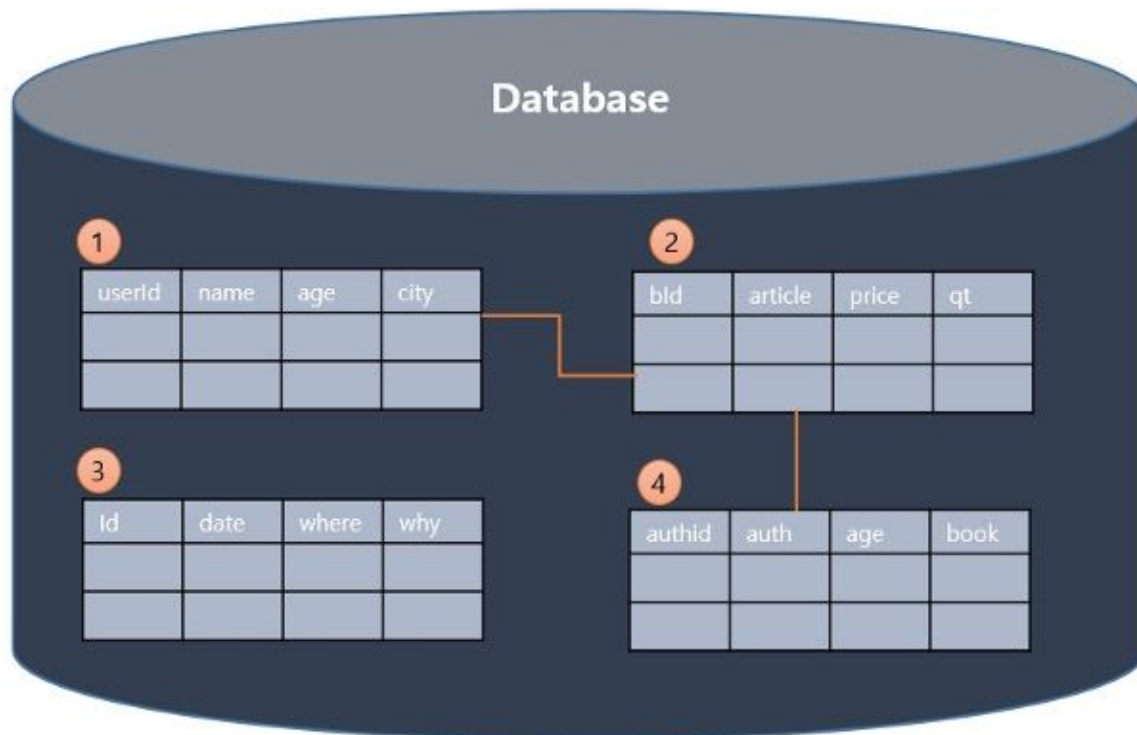

Tracker

More

So, is this a blockchain?

No, but bear with me.

Let's simplify things with an example and some figures, taking a [MySQL](#) database that stores some information in different tables.



An illustration of a MySQL database. | Image: Sinai Nday

With the above database, there are some limitations. It allows you to do some tasks, including:

1. Create, retrieve, update and delete (CRUD) operations.
2. Storing the same information twice.

But its limitations are many, including:

1. We can drop the entire database either mistakenly or intentionally.
2. We can't share sensitive information with others.
3. It's centralized, which means there's a single point of failure and a security

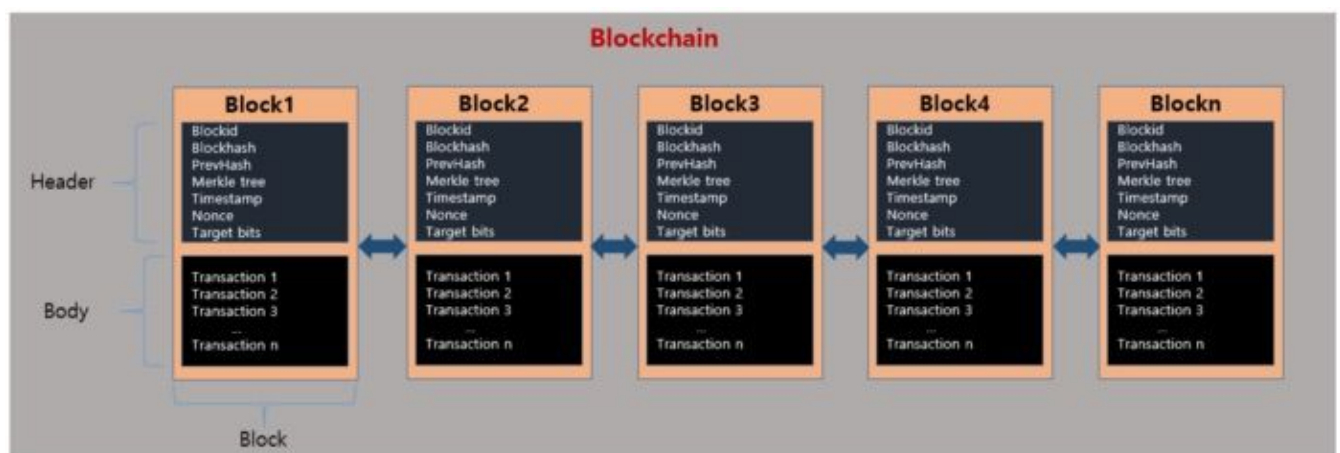
4. There's no way to trust everything that is stored in it.
5. Ill-intentioned people can compromise the database.
6. The database needs a manager.
7. The users don't have power over their own data.

We need something different that's transparent, reliable and independent from people. Something that's automatic, immutable, decentralized and indestructible. That's where blockchain kicks in.

A blockchain is a secure, trusted, decentralized database and network all in one.

How Does Blockchain Work?

In other words, a blockchain is a chain of blocks. These blocks are like tables in the database, but they can't be deleted or updated. The blocks contain information such as transactions, nonce, target bit, difficulty, timestamp, previous block hash, current block hash, Markle tree and block ID, and the blocks are cryptographically verified and chained up to form an immutable chain called a blockchain or a ledger.



An illustration of a blockchain. | Image: Sinai Nday

The same chain is then distributed to all the nodes (computers or miners) across the network via a P2P network.

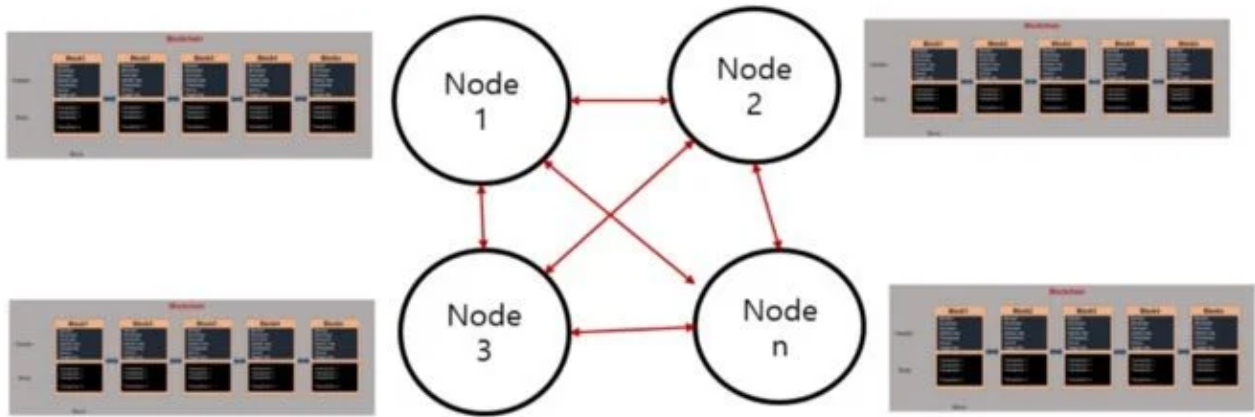


Illustration of a blockchain P2P network. | Image: Sinai Nday

Instead of a centralized database, all the transactions (data) that are shared across the nodes are contained in blocks, which are chained together to create the ledger. This ledger represents all the data in the blockchain. All the data in the ledger is secured by [cryptographic hashing](#) and [digital signature](#) and validated by a [consensus algorithm](#). Nodes on the network participate to ensure that all copies of the data distributed across the network are the same.

5 Key Concepts in the Blockchain Ecosystem

1. Cryptographic hash and digital signature.
2. Immutable ledger.
3. P2P network.
4. Consensus algorithm (PoW, PoS, PBFT, etc...).
5. Block validation (Mining, Forging, etc...).

I will explain these concepts more in detail as we go.

What Are the Benefits of Using Blockchain?

There are a number of [benefits to using blockchain](#), including the following upsides:

Blockchain removes the need for a [centralized authority](#) that oversees all transactions, instead distributing power among a network of computers. This setup places more control in the hands of all users and encourages a community approach.

Stronger Security

Users can bolster [blockchain security](#) through various methods. These include hashing techniques to encrypt individual blocks, [consensus mechanisms](#) to approve transactions and digital signatures to act as another layer of security. And the decentralized approach of blockchain enables users to quickly identify and address [bad actors](#).

Greater Transparency

While every computer that is part of a blockchain network can add and update information on the blockchain, the data itself is immutable. Each transaction is also stamped with the date and time, making it easier to track actions and [providing more transparency](#) for all users.

Improved Efficiency

Unlike traditional methods, blockchain removes the need for piles of paperwork in favor of digitized transactions. The technology also uses [smart contracts](#) to verify agreements, automating financial transactions.

Wider Access

Because blockchain runs on digital technology, it transcends geographic boundaries and removes the need to be near a financial hub. This gives broader audiences access to financial opportunities, especially those who may be shut out of traditional institutions.

Blockchain is not a silver bullet, so, it's best to use it when:

1. The data stored cannot be modified (proof of existence).
2. The data cannot be denied by its owner (non-repudiation).
3. You want [decentralization](#).
4. You want one source of truth.
5. You want high security.
6. You don't care about speed. For example, Bitcoin takes 10 minutes on average to validate a block of transactions. But some blockchains are faster because they use different consensus algorithms. We will talk about this later.

Blockchain Use Cases

Blockchain can be used in different sectors, including:

- **Real estate:** [Smart contracts](#) simplify applications and rental agreements, and blockchain makes it easier to [manage property assets](#).
- **Healthcare:** Blockchain can securely [record and store patient data](#), protecting against data breaches and prioritizing patient consent.
- **Finance:** With blockchain, users can conduct [cross-border payments](#), and immutable records help address money laundering and fraud.
- **Supply chain:** Companies can use blockchain to [track items](#) from vendors to customers, including verifying authenticity and original content creation.
- **Cybersecurity:** Due to its decentralized nature, blockchain is resistant to cyber attacks.
- **Government:** [Governments can use blockchain](#) to manage public records and make it easier to apply for foreign aid. This technology can also fortify the [voting process](#).
- **Education:** [Academic institutions](#) can store student data to verify grades and diplomas, and teachers can incentivize students with cryptocurrency rewards.
- **Retail:** [Retailers](#) can develop more efficient payment processes with



Q JOIN | LOG IN

chains.

- **Marketing:** [Marketers can use blockchain](#) to compile data on consumer interactions with ads, refining their ad campaigns over time.
- **Music:** Blockchain contributes to [music licensing](#) and intellectual property rights, and it can be used to deliver equitable royalty payments to musicians.

Blockchain Platforms to Know

- **Bitcoin:** [Bitcoin](#) supports the [Bitcoin cryptocurrency](#), which has the largest [market capitalization](#). The network itself uses [crypto mining](#) to verify transactions.
- **Ethereum:** [Ethereum](#) was the first platform to use smart contracts. In an event known as the [Ethereum Merge](#), the platform transitioned to a [proof-of-stake mechanism](#).
- **Hyperledger Fabric:** [Hyperledger Fabric](#) is an open-source platform that supports private transactions and can be tailored to various enterprise applications.
- **Corda:** [Corda](#) is another open-source platform that serves as a scalable technology to meet the needs of financial institutions.
- **EOS:** [EOS](#) is known for its user-friendliness and is used to build and maintain high-performing decentralized applications.
- **Cardano:** [Cardano](#) is based on peer-reviewed research and uses a proof-of-stake approach, serving as a secure and sustainable option for blockchain users.

MORE ON BLOCKCHAIN:

[22 Crypto Apps to Know](#)



Blockchain Types

Jobs

Companies

Articles


Tracker

More

- **Private:** Used only internally, and when we know the users (Hyperledger Fabric).
- **Public:** Everyone can see what is going on (Bitcoin, Ethereum).
- **Hybrid:** In case you want to combine the first two.

Create Your Own Blockchain

There are two ways to build a blockchain

1. The easiest way is to use a pre-built blockchain open-source like Ethereum (create distributed applications, [altcoins](#), [decentralized finance \(DeFi\)](#) and [non-fungible tokens \(NFTs\)](#), etc.), Fabric (configure a private blockchain), EOS or Cardano, so you don't have to deal with the core engine, which is difficult to implement.
2. If that doesn't fit your requirements, you can build one from scratch or fork, modify or improve an existing blockchain open-source code. For example, [Litecoin](#) and [Bitcoin cash](#) were forked from Bitcoin. This last method is tougher, more time-consuming and requires a lot of work and a strong team.

In this article, we will build a blockchain prototype from scratch so that you can thoroughly understand the blockchain's state machine.

MORE ON BLOCKCHAIN:

[What Is Layer 1 in Blockchain?](#)



How to Create a Blockchain From Scratch in Go

We're going to create the first baby blockchain in Go. You can also learn how to create it in [Python](#), and [JavaScript](#). These prototypes can help you understand the concepts we described earlier. We'll complete this in four steps:

Jobs

Companies

Articles


Tracker

More

1. Create a block.
2. Add the data (header and body) to the block.
3. Hash the block.
4. Chain the blocks together.

A block contains information mentioned earlier, but to simplify things we are going to remove some. Let's delve into the specifics.

If you aren't familiar with Go, try to familiarize yourself with the [basics](#), including functions, methods, data types, structures, flow controls and iterations.

1. Create a Block

We'll start with creating a folder with two files in it, `main.go` and `block.go`.

Folder structure

```
go // the folder
main.go // file 1
block.go // file 2
```

Main.go

```
// use the main package
package main

//import the fmt package
import (
    "fmt"
)

func main(){
    fmt.Println("I am building my first blockchain") // print this
    CreateBlock("The header will be shown here", "the body will be shown here")

}
```

If you run this program it will show an error because the `CreateBlock` function is not defined yet, so go ahead and create it in `block.go`.

```
package main

import (
    "fmt" // this will help us to print on the screen
)

func CreateBlock(Header, Body string){
    fmt.Println(Header ,"\n", Body) // Show me the block content
}
```

2. Add Data to Your Blocks

The beauty of Go is that you don't have to import or export functions, just declare them with capital letters, and Go will find them for you. Now, open a terminal and move to your created folder, and `run go build`, then run `.\go` on Windows, or `./go` on Linux and Macbook.

```

go > main.go > {} main > main
1 // use the main package
2 package main
3
4 //import the fmt package
5 import (
6     "fmt"
7 )
8
9 func main(){
10     fmt.Println("I am building my first blockchain") // print this
11     CreateBlock("The header will be shown here", "the body will be
12
13 }

```

TERMINAL DEBUG CONSOLE PROBLEMS OUTPUT

```

medium_articles\go> go build
medium_articles\go> .\go.exe
I am building my first blockchain
The header will be shown here
the body will be shown here
medium_articles\go>

```

Screenshot of the Go program that calls a function and passes a string of data. | Screenshot: Sinai Nday

We just created a simple Go program that calls a function and passes some string data. Let's add two more files, `blockchain.go` and `structures.go`. Now we have four files: `main.go`, `block.go`, `structures.go`, and `blockchain.go`

3. Hash Your Block

I will add to each line of code some comments for you to understand what I am doing.

Structures.go

```
package main //Import the main package
```

```
// A block contains this info:
```

```
type Block struct {
    Timestamp      int64 // the time when the block was created
    PreviousBlockHash []byte // the hash of the previous block
    MyBlockHash     []byte // the hash of the current block
    AllData         []byte // the data or transactions (body info)
}
```

```
// Prepare the Blockchain data structure :
```

```
type Blockchain struct {
    Blocks []*Block // remember a blockchain is a series of blocks
}
```

Block.go

```
package main
```

```
import (
    // We will need these libraries:
    "bytes"          // need to convert data into byte in order to be ser
    "crypto/sha256" //crypto library to hash the data
    "strconv"        // for conversion
    "time"           // the time for our timestamp
)
```

```
// Now let's create a method for generating a hash of the block
```

```
// We will just concatenate all the data and hash it to obtain the block has
```

```
func (block *Block) SetHash() {
    timestamp := []byte(strconv.FormatInt(block.Timestamp, 10))
    headers := bytes.Join([][]byte{timestamp, block.PreviousBlockHash, b
    hash := sha256.Sum256(headers)
```

}

// Create a function for new block generation and return that block

```
func NewBlock(data string, prevBlockHash []byte) *Block {
    block := &Block{time.Now().Unix(), prevBlockHash, []byte{}, []byte{}}
    block.SetHash()
    return block
}
```

/* let's now create the genesis block function that will return the first block

```
func NewGenesisBlock() *Block {
    return NewBlock("Genesis Block", []byte{}) // the genesis block is n
}
```

Blockchain.go

```
package main
```

// create the method that adds a new block to a blockchain

```
func (blockchain *Blockchain) AddBlock(data string) {
    PreviousBlock := blockchain.Blocks[len(blockchain.Blocks)-1] // the
    newBlock := NewBlock(data, PreviousBlock.MyBlockHash) // crea
    blockchain.Blocks = append(blockchain.Blocks, newBlock) // add
}
```

/* Create the function that returns the whole blockchain and add the genesis

```
func NewBlockchain() *Blockchain { // the function is created
    return &Blockchain{[]*Block{NewGenesisBlock()}} // the genesis block
}
```

Main

Jobs

Companies

Articles



Tracker

More


```
//Time to put everything together and test
package main
import (
    "fmt" // just for printing something on the screen
)
func main() {
    newblockchain := NewBlockchain() // Initialize the blockchain
    // create 2 blocks and add 2 transactions
    newblockchain.AddBlock("first transaction") // first block contain
    newblockchain.AddBlock("Second transaction") // second block contain
    // Now print all the blocks and their contents
    for _, block := range newblockchain.Blocks { // iterate on each block
        fmt.Printf("Hash of the block %x\n", block.MyBlockHash)
        fmt.Printf("Hash of the previous Block: %x\n", block.PreviousBlockHash)
        fmt.Printf("All the transactions: %s\n", block.AllData)
    } // our blockchain will be printed
}
```

Let's run it now, go build then `.\go`.

```
ndaysinai@Ndays-MacBook-Pro go % go build
ndaysinai@Ndays-MacBook-Pro go % ./go
Block ID : 0
Timestamp : 1610770863
Hash of the block : 58d43603575ae717828752ce1ebc5bcef86231de9a282a9122842c79dd73a4db
Hash of the previous Block :
All the transactions : Genesis Block
Block ID : 1
Timestamp : 1610770864
Hash of the block : 6465ae7dde0d7b5359518a26d0a198c6bcb8964b6c84d4a85ceaf3fdc9ea5fe0
Hash of the previous Block : 58d43603575ae717828752ce1ebc5bcef86231de9a282a9122842c79dd73a4db
All the transactions : first transaction
Block ID : 2
Timestamp : 1610770865
Hash of the block : 5962a66cff9c8d05ed3b02a901cf0ea539d715c911458acac7d5df293d582453
Hash of the previous Block : 6465ae7dde0d7b5359518a26d0a198c6bcb8964b6c84d4a85ceaf3fdc9ea5fe0
All the transactions : Second transaction
ndaysinai@Ndays-MacBook-Pro go %
```

Result. | Screenshot: Sinai Nday

Voila. Easy, right?

Oops, our blocks in the blockchain don't have any IDs and timestamps. So, let's add that information by modifying the `main.go` file, add these two lines in the `for` loop :

```
fmt.Printf("Block ID : %d \n", i)
```

fmt

```
//Time to put everything together and test
```

```
package main
```

```
import (
```

```
    "fmt" // just for printing something on the screen
```

```
)
```

```
func main() {
```

```
    newblockchain := NewBlockchain() // Initialize the blockchain
```

```
    // create 2 blocks and add 2 transactions
```

```
    newblockchain.AddBlock("first transaction") // first block containi
```

```
    newblockchain.AddBlock("Second transaction") // second block contain
```

```
    // Now print all the blocks and their contents
```

```
    for i, block := range newblockchain.Blocks { // iterate on each bloc
```

```
        fmt.Printf("Block ID : %d \n", i)
```

```
        fmt.Printf("Timestamp : %d \n", block.Timestamp+int64(i))
```

```
        fmt.Printf("Hash of the block : %x\n", block.MyBlockHash)
```

```
        fmt.Printf("Hash of the previous Block : %x\n", block.PreviousBlockHash)
```

```
        fmt.Printf("All the transactions : %s\n", block.AllData)
```

```
    } // our blockchain will be printed
```

```
}
```

Let's save the code and run it again, go build then `./go`.

```
ndaysina@Ndays-MacBook-Pro go % go build
ndaysina@Ndays-MacBook-Pro go % ./go
Hash of the block : e44c950dca8754d34e2849245e61d3f0da3e8a536f293e3ea644dba57a616caf
Hash of the previous Block :
All the transactions : Genesis Block
Hash of the block : e0b61ae2962d278966042e9b4c6be9c4f365c164cf1ad0d13cf59687e4b5ca09
Hash of the previous Block : e44c950dca8754d34e2849245e61d3f0da3e8a536f293e3ea644dba57a616caf
All the transactions : first transaction
Hash of the block : a21a76e34b0aec35dc64242dd9797580f3d2567446a72fb1e55ff6f96e0364ee
Hash of the previous Block : e0b61ae2962d278966042e9b4c6be9c4f365c164cf1ad0d13cf59687e4b5ca09
All the transactions : Second transaction
ndaysina@Ndays-MacBook-Pro go %
```

Final result of your blockchain. | Image: Sinai Nday

As you can see, the blockchain is well structured. Except for the genesis block, each block contains its hash and the hash of the previous block, which makes it immutable. If the data in the block is altered, the hash will automatically change and the block will be discarded. The genesis block doesn't have any previous hash because it's the first one. There is no previous block.

And that's it. Congratulations, you've just created your first [baby blockchain](#) in Go.

We will gradually fulfill the blockchain requirements. Blockchain is a masterpiece that needs to be engineered the way it's supposed to be.

In blockchain, there are also two common roles — blockchain engineer and [blockchain developer](#). A blockchain engineer is a professional who thoroughly understands the principles of blockchain, security and software engineering for designing, developing, maintaining, testing and evaluating the blockchain core engines and software. A blockchain developer is a professional who builds software on top of the blockchain called decentralized applications.

Most blockchain developers use open [blockchain platforms](#) and frameworks like Ethereum, Hyperledger Fabric and EOS. Now that you have the basics, it's up to you to decide which one you want to be.

Frequently Asked Questions

How much does it cost to create a blockchain?

Creating a blockchain can cost [anywhere from \\$15,000 to \\$50,000](#), depending on the particular task and the complexity of the platform or application.



Is a blockchain easy to create?

It depends on the type of blockchain platform or application being built. In general, creating a blockchain takes time and requires knowledge of basic blockchain concepts, certain programming languages and digital security best practices.



What programming language is needed for blockchain?

Different blockchains use various programming languages, with some popular options being C++, JavaScript, Rust, Go and Python.



Recent Blockchain Articles



[What Is Polymarket? How the Prediction Market Platform Works.](#)



[Blockchain in Finance: What It Is and How It's Used](#)



[Why We Need 'Know Your Agent' Standards Now](#)



Built In is the online community for startups and tech companies. Find startup jobs, tech news and events.



About

[Our Story](#)

[Careers](#)

[Our Staff Writers](#)

[Content Descriptions](#)

Get Involved

[Recruit With Built In](#)

[Become an Expert Contributor](#)

Resources

[Customer Support](#)

[Share Feedback](#)

[Report a Bug](#)

[Tech Job Tools + Career Resources](#)

[Browse Jobs](#)

[Tech A-Z](#)

Tech Hubs

Our Sites

[Learning Lab User Agreement](#)

[Accessibility Statement](#)

[Copyright Policy](#)

[Privacy Policy](#)

[Terms of Use](#)

[Your Privacy Choices/Cookie Settings](#)

[CA Notice of Collection](#)

© Built In 2026